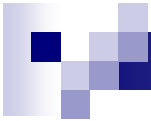


Производные типы данных в MPI

- Сообщение в MPI представляет собой массив **однотипных** данных, элементы которого расположены в **последовательных** ячейках памяти.
- Иногда возникает необходимость в пересылке разнотипных данных или фрагментов массивов, содержащих элементы, расположенные не в последовательных ячейках. Так бывает, например, при программировании вычислений, связанных с операциями с матрицами и векторами, а также в других ситуациях.
- В MPI-программе для пересылки таких данных необходимо создать производный тип данных.



Производные типы данных в MPI

- Производные типы данных создаются во время выполнения программы (а не во время ее трансляции), как правило, перед их использованием.

- Создание типа - процесс, который состоит из двух шагов:

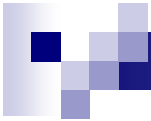
- ☐ конструирование типа
- ☐ регистрация типа:

```
int MPI_Type_commit ( MPI_Datatype *type );
```

- После завершения работы с производным типом, он аннулируется.

```
int MPI_Type_free ( MPI_Datatype *type );
```

- При этом все производные от него типы остаются и могут использоваться дальше, пока и они не будут уничтожены.



Производные типы данных в MPI

- Производный тип данных в MPI характеризуется последовательностью базовых типов и набором целочисленных значений смещения – картой типа
- Смещения отсчитываются относительно начала буфера обмена и определяют те элементы данных, которые будут участвовать в обмене.
- Не требуется, чтобы они были упорядочены (по возрастанию или по убыванию).
- Отсюда следует, что порядок элементов данных в производном типе может отличаться от исходного и, кроме того, один элемент данных может появляться в новом типе многократно.
- Часть карты типа с указанием только типов значений называется в MPI сигнатурой типа



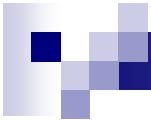
Карта типа данных

- **Пример.** Пусть в сообщение должны входить значения переменных:

```
double a; /* адрес 24 */  
double b; /* адрес 40 */  
int n; /* адрес 48 */
```

Тогда производный тип для описания таких данных должен иметь карту типа следующего вида:

```
{ (MPI_DOUBLE, 0),  
  (MPI_DOUBLE, 16),  
  (MPI_INT,    24)  
}
```



Карта типа данных

- Для производных типов данных в MPI используется следующий ряд новых понятий:

- *нижняя граница* типа:

$$lb (TypeMap) = \min_j (disp_j)$$

- *верхняя граница* типа:

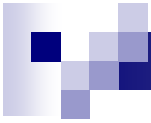
$$ub (TypeMap) = \max_j (disp_j + sizeof(type_j)) + \Delta$$

- *протяженность* типа (размер памяти в байтах, который нужно отводить для одного элемента производного типа):

$$extent (TypeMap) = ub (TypeMap) - lb (TypeMap)$$

- *размер* типа данных - это число байтов, которые занимают данные.

- Различие в значениях протяженности и размера состоит в величине округления для выравнивания адресов.



Карта типа данных

- Для получения значения протяженности и размера типа в MPI предусмотрены функции:

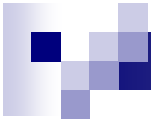
```
int MPI_Type_extent ( MPI_Datatype type, MPI_Aint *extent );  
int MPI_Type_size ( MPI_Datatype type, MPI_Aint *size );
```

- Определение нижней и верхней границ типа может быть выполнено при помощи функций:

```
int MPI_Type_lb ( MPI_Datatype type, MPI_Aint *disp );  
int MPI_Type_ub ( MPI_Datatype type, MPI_Aint *disp );
```

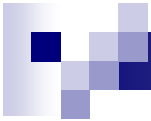
- Важной и необходимой при конструировании производных типов является функция получения адреса переменной:

```
int MPI_Address ( void *location, MPI_Aint *address );
```



Способы конструирования производных типов данных

- Способы создания нового типа из элементов одного и того же базового типа (чаще всего – подмножества элементов массива)
 - *непрерывный* – непрерывный набор элементов базового типа определяется как новый производный тип;
 - *векторный* – создание нового типа как набора элементов существующего типа с регулярными промежутками по памяти;
 - *индексный* – промежутки между элементами могут иметь нерегулярный характер.
- Наиболее общий способ конструирования производных типов – *структурный* – позволяет объединить разнотипные данные. Данный способ конструирования требует явное указание карты создаваемого типа данных.



Непрерывный способ конструирования

- Функция

```
int MPI_Type_contiguous( int count,  
                        MPI_Datatype oldtype,  
                        MPI_Datatype *newtype );
```

создаёт новый тип **newtype** из **count** элементов исходного типа **oldtype**.

- *Пример:*

```
MPI_Datatype newtype;  
int MPI_Type_contiguous(10, MPI_INT, &newtype);
```




Векторный способ конструирования

- Функция

```
int MPI_Type_vector( int count, int blocklen,  
                    int stride,  
                    MPI_Datatype oldtype,  
                    MPI_Datatype *newtype );
```

создаёт новый тип **newtype** из **count** блоков по **blocklen** элементов исходного типа **oldtype**,
stride – количество элементов расположенных между соседними блоками

- Расстояние между блоками также можно задать не в элементах, а в байтах

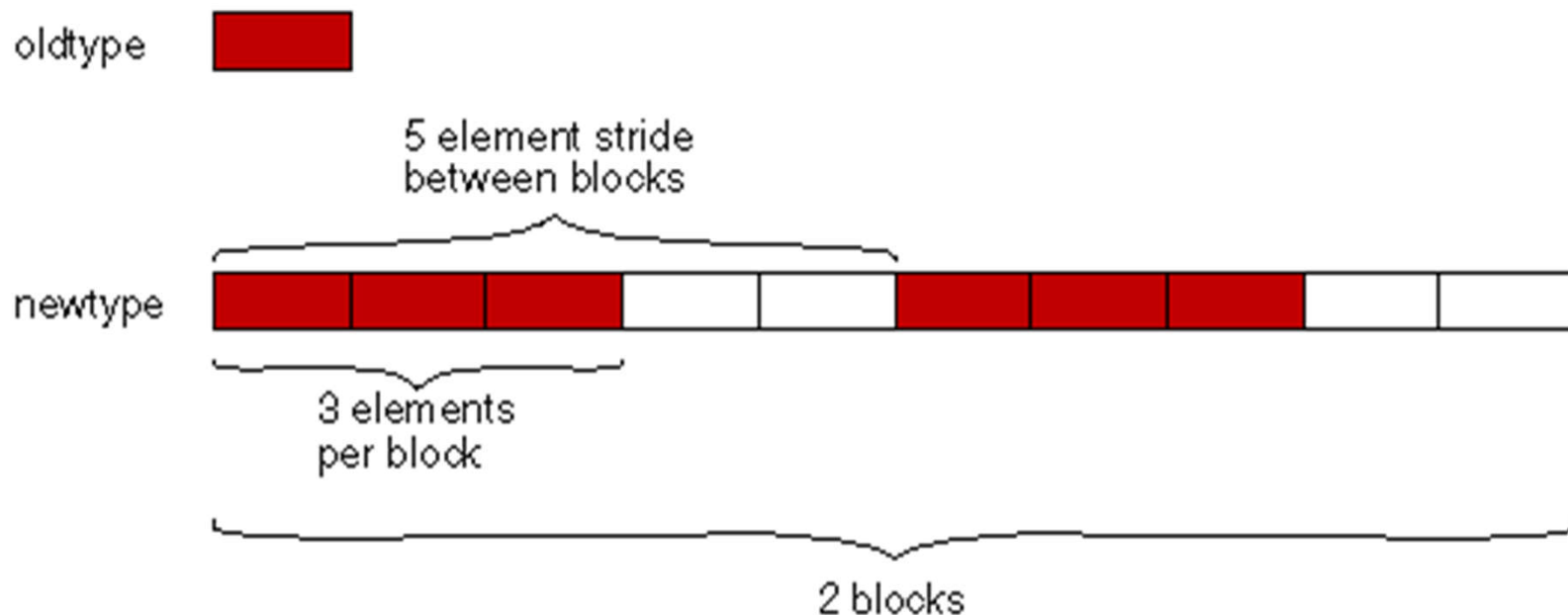
```
int MPI_Type_hvector( int count, int blocklen,  
                     MPI_Aint stride,  
                     MPI_Datatype oldtype,  
                     MPI_Datatype *newtype );
```

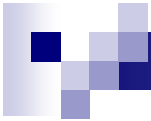
Векторный способ конструирования

- Вызов

```
MPI_Type_vector( 2, 3, 5, MPI_INT, &block_type);
```

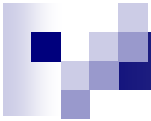
создаст тип данных, указанный на рисунке





Векторный способ конструирования

- Конструирование типа для выделения половины (только четных или только нечетных) строк матрицы размером $N \times N$:
`MPI_Type_vector( n/2, n, 2*n, MPI_INT, &StripRowType );`
- Конструирование типа для выделения столбца матрицы размером $N \times N$:
`MPI_Type_vector ( n, 1, n, MPI_INT, &ColumnType );`
- Конструирование типа для выделения главной диагонали матрицы размером $N \times N$:
`MPI_Type_vector ( n, 1, n + 1, MPI_INT, &DiagType );`



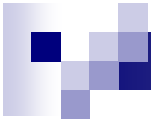
Описание подмассивов в MPI-2

- Тип данных, соответствующий подмассиву многомерного массива, можно создать с помощью функции

```
int MPI_Type_create_subarray ( int ndims, int *sizes,  
                              int *subsizes, int *starts,  
                              int order,  
                              MPI_Datatype oldtype,  
                              MPI_Datatype *newtype )
```

где

- **ndims** — размерность массива;
- **sizes** — количество элементов типа **oldtype** в каждом измерении полного массива;
- **subsizes** — количество элементов типа **oldtype** в каждом измерении подмассива;
- **starts** — стартовые координаты подмассива в каждом измерении;
- **order** — флаг, задающий переупорядочение.



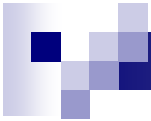
Индексный способ конструирования

- При индексном способе конструирования производного типа данных в MPI используются функции:

```
int MPI_Type_indexed ( int count, int blocklens[],  
                      int indices[],  
                      MPI_Datatype oldtype,  
                      MPI_Datatype *newtype ),
```

где

- **count** – количество блоков,
- **blocklens** – количество элементов в каждом блоке,
- **indices** – смещение каждого блока от начала типа (в количестве элементов исходного типа),
- **oldtype** - исходный тип данных,
- **newtype** - новый определяемый тип данных.



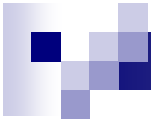
Индексный способ конструирования

- Функция:

```
int MPI_Type_hindexed ( int count, int blocklens[],  
                        MPI_Aint indices[],  
                        MPI_Data_type oldtype,  
                        MPI_Datatype *newtype )
```

отличается от предыдущей тем, что смещения задаются в байтах

- Индексный способ конструирования позволяет задавать разные промежутки по памяти

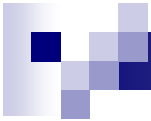


Индексный способ конструирования

- Конструирование типа для описания верхней треугольной матрицы размером $N \times N$:

```
for ( i = 0; i < N; ++i)
{
    blocklens[i] = n - i;
    indices[i] = i * n + i;
}
MPI_Type_indexed ( n, blocklens, indices, MPI_INT,
                  &UTMatrixType );
```

- В стандарте MPI-2 предусмотрена функция `MPI_Type_create_indexed_block` для индексного способа конструирования типов с блоками одинакового размера



Структурный способ конструирования

- Структурный способ конструирования является самым общим методом конструирования производного типа данных при явном задании соответствующей карты типа:

```
int MPI_Type_struct ( int count, int blocklens[],  
                     MPI_Aint indices[],  
                     MPI_Data_type oldtypes[],  
                     MPI_Datatype *newtype ),
```

где

- **count** – количество блоков,
- **blocklens** – количество элементов в каждом блоке,
- **indices** – смещение каждого блока от начала типа (в количестве элементов исходного типа),
- **oldtypes** - исходные типы данных в каждом блоке,
- **newtype** - новый определяемый тип данных.



Пример

```
#include "mpi.h"
struct newtype {
    float a;
    float b;
    int n;
};
int main(int argc, char *argv[])
{
    int myrank;
    MPI_Datatype NEW_MESSAGE_TYPE;
    int block_lengths[3];
    MPI_Aint displacements[3];
    MPI_Aint addresses[4];
    MPI_Datatype typelist[3];
    int blocks_number;
    struct newtype indata;
    int tag = 0;
    MPI_Status status;
```



Пример

```
MPI_Init( &argc, &argv );
MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
typelist[0] = MPI_FLOAT;
typelist[1] = MPI_FLOAT;
typelist[2] = MPI_INT;
block_lengths[0] = block_lengths[1] = block_lengths[2] = 1;
MPI_Address( &indata, &addresses[0] );
MPI_Address( &(indata.a), &addresses[1] );
MPI_Address( &(indata.b), &addresses[2] );
MPI_Address( &(indata.n), &addresses[3] );
displacements[0] = addresses[1] - addresses[0];
displacements[1] = addresses[2] - addresses[0];
displacements[2] = addresses[3] - addresses[0];
blocks_number = 3;
MPI_Type_struct( blocks_number, block_lengths, displacements,
                typelist, &NEW_MESSAGE_TYPE );
MPI_Type_commit( &NEW_MESSAGE_TYPE );
```



Пример

```
if (myrank == 0) {
    indata.a = 3.14159;
    indata.b = 2.71828;
    indata.n = 2002;
    MPI_Send( &indata, 1, NEW_MESSAGE_TYPE, 1, tag,
              MPI_COMM_WORLD );
    printf( "Process %i send: %f %f %i\n", myrank, indata.a,
            indata.b, indata.n );
}
else {
    MPI_Recv( &indata, 1, NEW_MESSAGE_TYPE, 0, tag,
              MPI_COMM_WORLD, &status );
    printf( "Process %i received: %f %f %i, status %s\n", myrank,
            indata.a, indata.b, indata.n, status.MPI_ERROR );
}
MPI_Type_free( &NEW_MESSAGE_TYPE );
MPI_Finalize();
return 0;
}
```



Упаковка и распаковка данных

- Явный способ сборки и разборки сообщений, в которые могут входить значения разных типов и располагаемых в разных областях памяти:

```
int MPI_Pack ( void *data, int count, MPI_Datatype type,  
              void *buf, int bufsize, int *bufpos,  
              MPI_Comm comm );
```

где

- **data** – буфер памяти с элементами для упаковки,
- **count** – количество элементов в буфере,
- **type** – тип данных для упаковываемых элементов,
- **buf** - буфер памяти для упаковки,
- **buflen** – размер буфера в байтах,
- **bufpos** – позиция для начала записи в буфер (в байтах от начала буфера),
- **comm** - коммуникатор для упакованного сообщения.

Упаковка и распаковка данных

данные для упаковки



Буфер упаковки



bufpos



MPI_Pack

данные для упаковки



Буфер упаковки



bufpos

а) упаковка данных

буфер для распаковываемых данных



Распаковываемый буфер



bufpos



MPI_Unpack

данные после распаковки

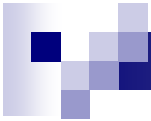


Распаковываемый буфер



bufpos

б) распаковка данных



Упаковка и распаковка данных

- Для определения необходимого размера буфера для упаковки может быть использована функция:

```
int MPI_Pack_size ( int count, MPI_Datatype type,  
                   MPI_Comm comm,  
                   int *size);
```

- После упаковки всех необходимых данных подготовленный буфер может быть использован в функциях передачи данных с указанием типа *MPI_PACKED*,

- После получения сообщения с типом *MPI_PACKED* данные могут быть распакованы при помощи функции:

```
int MPI_Unpack ( void *buf, int bufsize, int *bufpos,  
                void *data, int count, MPI_Datatype type,  
                MPI_Comm comm);
```



Упаковка и распаковка данных

- Вызов функции *MPI_Pack* осуществляется последовательно для упаковки всех необходимых данных. Если в сообщение должны входить данные

```
double a /* адрес 24 */;  
double b /* адрес 40 */;  
int n /* адрес 48 */;
```

то для их упаковки необходимо выполнить:

```
bufpos = 0;  
MPI_Pack(a, 1, MPI_DOUBLE, buf, buflen, &bufpos, comm);  
MPI_Pack(b, 1, MPI_DOUBLE, buf, buflen, &bufpos, comm);  
MPI_Pack(n, 1, MPI_INT, buf, buflen, &bufpos, comm);
```

- Для распаковки упакованных данных необходимо выполнить:

```
bufpos = 0;  
MPI_Unpack(buf, buflen, &bufpos, a, 1, MPI_DOUBLE, comm);  
MPI_Unpack(buf, buflen, &bufpos, b, 1, MPI_DOUBLE, comm);  
MPI_Unpack(buf, buflen, &bufpos, n, 1, MPI_INT, comm);
```



Пример

```
#include "mpi.h"
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int myrank;
```

```
    float a, b;
```

```
    int n;
```

```
    int root = 0;
```

```
    char buffer[100];
```

```
    int position;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
```




Пример

```
if (myrank == root) {  
    printf("Enter a, b, and n\n");  
    scanf("%f %f %i", &a, &b, &n);  
    position = 0;  
    MPI_Pack( &a, 1, MPI_FLOAT, &buffer, 100, &position,  
             MPI_COMM_WORLD);  
    MPI_Pack( &b, 1, MPI_FLOAT, &buffer, 100, &position,  
             MPI_COMM_WORLD);  
    MPI_Pack( &n, 1, MPI_INT, &buffer, 100, &position,  
             MPI_COMM_WORLD);  
    MPI_Bcast(&buffer, 100, MPI_PACKED, root, MPI_COMM_WORLD);  
}
```



Пример

```
else {  
    MPI_Bcast( &buffer, 100, MPI_PACKED, root, MPI_COMM_WORLD );  
    position = 0;  
    MPI_Unpack( &buffer, 100, &position, &a, 1, MPI_FLOAT,  
                MPI_COMM_WORLD );  
    MPI_Unpack( &buffer, 100, &position, &b, 1, MPI_FLOAT,  
                MPI_COMM_WORLD );  
    MPI_Unpack( &buffer, 100, &position, &n, 1, MPI_INT,  
                MPI_COMM_WORLD );  
    printf( "Process %i received a=%f, b=%f, n=%i\n",  
            myrank, a, b, n );  
}  
MPI_Finalize();  
return 0;  
}
```